

Sentinel AI Security Audit Report

Audit Metadata

- **Filename:** vuln.exe
- **SHA-256 Checksum:** 5c7a223907e8926d99b4f20b60f4bc59a15f9828e22a2e3d7c176c90294f0d05
- **Verdict:** VULNERABLE
- **Risk Level:** CRITICAL
- **Total Functions Evaluated:** 18

Executive Summary

The Sentinel AI binary audit of `vuln.exe` has identified a **CRITICAL** security vulnerability within the analyzed codebase. Our proprietary GNN-based analysis engine, utilizing the GATv2 (Graph Attention Network) architecture, flagged a high-risk memory safety violation in the function `vuln\_c\_func`.

The audit reveals a classic stack-based buffer overflow vector. This vulnerability allows an attacker to provide crafted input that exceeds the allocated memory boundaries of a local stack buffer, leading to the corruption of adjacent memory, including the saved Base Pointer (RBP) and the Return Address. Exploitation of this flaw could lead to arbitrary code execution (ACE), allowing a threat actor to gain full control over the host process. Immediate remediation is required to mitigate the risk of system compromise.

Analysis Metrics

- **Analysis Engine:** GNN-Compiler Wrapper (GATv2 Model)
- **Total Functions Scanned:** 18
- **Flagged Functions:** 1
- **Model Confidence Score:** 57.63%
- **Primary Threat Classification:** CWE-119 (Improper Restriction of Operations within the Bounds of a Memory Buffer)
- **Secondary Threat Classification:** CWE-787 (Out-of-bounds Write)

Detailed Findings

Finding 1: Function `vuln\_c\_func`

- **Threat Type:** Stack-Based Buffer Overflow / Memory Safety Violation
- **Confidence:** 57.63%
- **Rule Violation:** SEI CERT C Rule ARR30-C ("Do not form or use out-of-bounds pointers or array subscripts") and STR31-C ("Guarantee that storage for strings has sufficient space for character data and the null terminator").

Technical Breakdown:

The disassembly reveals a critical flaw in the stack frame management and string handling of `vuln\_c\_func`.

1. **Stack Allocation:** The function allocates `0x30` (48 bytes) for its stack frame (`SUB RSP, 0x30`).
2. **Buffer Definition:** A local buffer is located at `[RBP - 0x10]`. This indicates a destination buffer size of only 16 bytes before it begins overwriting the saved frame pointer.
3. **Vulnerable Sink:** The function calls `strcpy` (at address `0x140003258`). The source string is passed via `RCX` (stored at `[RBP + 0x10]`), and the destination is the 16-byte stack buffer.
4. **Exploit Vector:** Because `strcpy` does not perform bounds checking, any input string longer than 15 characters (plus the null terminator) will overflow the buffer. This will overwrite the `RBP` and the `Return Address` on the stack, enabling a "Return-to-libc" or "Return-Oriented Programming" (ROP) attack.

Disassembly Analysis:

Basic Block 1:

```
MOV RDX, qword ptr [RBP + 0x10] ; Load source string pointer
LEA RAX, [RBP + -0x10]          ; Load address of 16-byte destination buffer
MOV RCX, RAX                    ; Set destination for strcpy
CALL 0x140003258                ; <--- CRITICAL VULNERABILITY: strcpy
```

Remediation & Secure Code Fix:

To remediate this, the developer must replace the unsafe `strcpy` call with a bounds-checked alternative and validate the length of the input data.

Insecure Implementation (Conceptual):

```
void vuln_c_func(char *input) {
    char buffer[16];
    if (input != NULL) {
        strcpy(buffer, input); // VULNERABLE
        printf("%s", buffer);
    }
}
```

Secure Implementation (Recommended):

```
#include <string.h>
#include <stdio.h>

#define MAX_BUF_SIZE 16

void secure_c_func(const char *input) {
    char buffer[MAX_BUF_SIZE];

    if (input == NULL) {
        return;
    }

    // Use strncpy to limit copy length to the size of the destination buffer
    // Ensure the string is null-terminated manually
    strncpy(buffer, input, MAX_BUF_SIZE - 1);
    buffer[MAX_BUF_SIZE - 1] = '\0';

    // Alternatively, use C11 strcpy_s for safer bounds checking
    // errno_t err = strcpy_s(buffer, sizeof(buffer), input);

    printf("%s", buffer);
}
```

General Mitigations

To harden the `vuln.exe` binary against future memory corruption exploits, the following compiler-level and system-level protections should be implemented:

1. **Stack Canaries (Stack Cookies):** Enable the `/GS` flag (MSVC) or `-fstack-protector-all` (GCC/Clang) to insert a "canary" value before the return address. If the canary is overwritten by a buffer overflow, the program will terminate before the return address is used.
2. **Address Space Layout Randomization (ASLR):** Ensure the binary is compiled with `/DYNAMICBASE` to randomize the memory locations of the stack, heap, and libraries, making it difficult for attackers to predict target addresses for ROP chains.
3. **Data Execution Prevention (DEP / NX):** Mark the stack and heap as non-executable (`/NXCOMPAT`) to prevent the execution of injected shellcode.
4. **Control Flow Guard (CFG):** Implement CFG to provide platform-level security that prevents indirect call hijacking.
5. **Static and Dynamic Analysis:** Integrate SAST (Static Analysis Security Testing) into the CI/CD pipeline to catch unsafe function usage (like `strcpy`, `gets`, `sprintf`) before deployment.

End of Report

Sentinel AI Security Audit - 2023